Understanding Pipelining Hazards

BLG 483E – Artificial Intelligence

salihsefer36

May 10, 2026

Introduction to CPU Pipelining

- Instruction-level parallelism (ILP) is exploited via pipelining.
- A pipeline divides instruction execution into k discrete stages: IF, ID, EX, MEM, WB.
- Throughput approaches 1 instruction per clock cycle (IPC ≈ 1) under ideal conditions.
- Hazards are structural, temporal, or data-flow conflicts that prevent the pipeline from advancing at every clock edge.
- Hazard penalties manifest as pipeline stalls (bubbles) or incorrect architectural state.

Data Hazards

- Data hazards arise from read-after-write (RAW), write-after-read (WAR), and write-after-write (WAW) dependences between instructions.
- RAW (true dependence): a consumer instruction attempts to read a register operand before the producer instruction completes its WB stage.
- WAR (anti-dependence): a later instruction writes a destination before a prior instruction finishes reading the same register.
- WAW (output dependence): two instructions write the same architectural register; the second write must dominate the first in program order.
- Detection requires a hazard detection unit that monitors the ID/EX, EX/MEM, and MEM/WB pipeline registers on every cycle.
- Mitigation: stall insertion (pipeline interlock) or operand forwarding (bypassing) from EX/MEM or MEM/WB to the EX stage ALU inputs.

Structural Hazards

- Structural hazards occur when multiple instructions simultaneously require the same hardware resource.
- Classic example: a unified memory with no instruction/data cache separation forces a stall when MEM and IF stages both require access.
- Register-file write-port conflicts arise when two instructions attempt to write-back concurrently in single-port implementations.
- In floating-point pipelines, multi-cycle functional units (dividers, square-root units) can create write-back contention.
- Resolution strategies: resource duplication, resource time-sharing, or pipeline stage reordering with compiler scheduling.

Control Hazards

- Control hazards (branch hazards) arise because branch outcome and target address are resolved late in the pipeline (EX or MEM stage).
- In a 5-stage pipeline, a taken branch incurs a 2-cycle penalty if no mitigation is applied (instructions at IF and ID must be flushed).
- Static prediction strategies: predict-not-taken, predict-taken, BTFN (backward-taken, forward-not-taken).
- Dynamic prediction: 1-bit and 2-bit saturating counters in a Branch Prediction Buffer (BPB) indexed by the lower bits of the PC.
- Branch Target Buffer (BTB) stores predicted target addresses to eliminate target-address computation latency.
- Delayed branching exposes one branch delay slot filled by the compiler with a useful instruction or a `NOP`.

Hazard Mitigation Summary

- **Stall (interlock):** The hazard detection unit asserts a stall signal, holding PC and IF/ID registers constant while inserting a bubble (NOP) into the ID/EX register.
- **Forwarding (bypassing):** Multiplexers route the result from the EX/MEM or MEM/WB register directly to the ALU input, reducing RAW stalls from 2 cycles to 0 for most cases.
- **Load-use hazard:** A load followed immediately by a dependent instruction cannot be fully resolved by forwarding alone; one stall cycle remains unavoidable without out-of-order execution.
- **Branch delay slots:** RISC ISAs (MIPS, SPARC) expose one slot post-branch; the instruction in this slot always executes.
- **Compiler scheduling:** Static reordering eliminates hazards at compile time by inserting independent instructions between dependent pairs.

References


Patterson, D. A. and Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2014.


Hennessy, J. L. and Patterson, D. A. *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.